DWES - Cómo añadir cualquier endpoint

Manual paso a paso para Spring Boot REST

Pensado para consulta offline durante el examen

0. La regla que evita la mayoría de errores

SIEMPRE Antes de escribir código, traduce el endpoint a 5 datos: método HTTP, ruta, entrada, salida y errores.

Pregunta	Ejemplo
¿Método HTTP?	GET
¿Ruta?	/api/libros/{id}
¿Qué recibe?	id por @PathVariable
¿Qué devuelve?	Libro
¿Qué errores?	404 si no existe

REGLA Controller = HTTP. Service = lógica. No recorras listas, valides reglas ni calcules resultados en el Controller.

1. ¿Qué archivos tengo que tocar?

Lo que pide el enunciado	Archivos habituales	Busca en Biblioteca Examen
GET simple / lista	Service + ServiceImpl + Controller	listar() / @GetMapping
GET por ID	Service + ServiceImpl + Controller	obtenerPorId
POST crear	DTO/model + Service + ServiceImpl + Controller	RESPONSEENTITY CREATED
PUT modificar	DTO/model + Service + ServiceImpl + Controller	modificar
DELETE	Service + ServiceImpl + Controller	eliminar
Filtro ?campo=valor	Service + ServiceImpl + Controller	VARIOS FILTROS OPCIONALES
Recurso relacionado	dos Services + Controller	listarPorCategoria
Respuesta calculada	DTO + Service específico + Controller	RESULTADO CALCULADO
Nueva regla / conflicto	ServiceImpl + excepción existente/nueva	REGLA AL BORRAR / ConflictoException
Configuración	application.properties + clase config	BibliotecaPropiedades

2. Procedimiento universal para añadir un endpoint

1. Escribe en papel o comentario el método, la ruta, qué recibe, qué devuelve y qué errores puede producir.
2. Decide si la operación pertenece a un Service que ya existe o necesita uno nuevo.
3. Añade la firma del método a la interfaz Service si hace falta.
4. Implementa la lógica en ServiceImpl. Aquí van búsquedas, filtros, validaciones, cálculos y reglas de negocio.
5. Añade el método del Controller con la anotación HTTP correcta. El Controller debe delegar en el Service.
6. Si necesitas devolver datos que no son una entidad, crea un DTO.
7. Si una condición debe producir 400/404/409, lanza la excepción desde el Service.
8. Arranca y prueba SOLO ese endpoint. No esperes a terminar todo el examen.
9. Si funciona, prueba también al menos un caso de error.
10. Solo si el enunciado lo exige, documenta con Swagger/OpenAPI y añade la petición a Postman.

3. GET - Listar todos los recursos

CUÁNDO	El enunciado dice listar, obtener todos, consultar catálogo, mostrar todos...
BUSCA	Ctrl + Shift + F -> public List<Libro> listar. Referencia: Biblioteca Decroly Examen.

```
// Service
List<Libro> listar();

// ServiceImpl
@Override
public List<Libro> listar() {
   return libros;
}

// Controller
@GetMapping
public List<Libro> listar() {
   return libroServicio.listar();
}
```

RESPUESTA	Normalmente 200 OK. Si no hay elementos, devuelve [] y sigue siendo 200.

4. GET - Buscar un recurso por ID

CUÁNDO	El enunciado pide consultar un recurso concreto mediante su identificador.
BUSCA	Ctrl + Shift + F -> obtenerPorId. Referencia: Biblioteca Decroly Examen.

```
// Service
Libro obtenerPorId(Long id);

// ServiceImpl
@Override
public Libro obtenerPorId(Long id) {
   for (Libro libro : libros) {
      if (libro.getId().equals(id)) {
         return libro;
      }
   }
   throw new RecursoNoEncontradoException(
         "No existe ningún libro con el ID " + id);
}

// Controller
@GetMapping("/{id}")
public Libro obtenerPorId(@PathVariable Long id) {
   return libroServicio.obtenerPorId(id);
}
```

ERRORES	Existe -> 200. ID correcto pero recurso inexistente -> 404.

5. POST - Crear un recurso

CUÁNDO	El cliente envía datos para crear un elemento nuevo.
BUSCA	Ctrl + Shift + F -> RESPONSEENTITY CREATED + LOCATION. Referencia: Biblioteca Decroly Examen.

11. Recibe los datos con @RequestBody. Usa DTO si la entrada no debe contener campos como id o disponibles.
12. Valida datos en ServiceImpl.
13. Comprueba duplicados o relaciones necesarias.
14. Asigna ID en el servidor.
15. Guarda el objeto en la lista.
16. Devuelve 201 Created. Si lo piden o quieres seguir el patrón completo, añade Location.

```
// Controller
@PostMapping
public ResponseEntity<Libro> crear(@RequestBody LibroPeticionDto dto) {
   Libro creado = libroServicio.crear(dto);
   URI direccion = URI.create("/api/libros/" + creado.getId());
   return ResponseEntity.created(direccion).body(creado);
}
```

CÓDIGOS	201 creado. 400 datos inválidos. 409 campo único duplicado. Según la regla, una relación inexistente puede tratarse como 400 o 404; sigue el criterio del enunciado/profesor.

6. PUT - Modificar un recurso completo

CUÁNDO	El enunciado dice modificar/actualizar un recurso existente enviando sus nuevos datos.
BUSCA	Ctrl + Shift + F -> public Libro modificar. Referencia: Biblioteca Decroly Examen.

```
// ServiceImpl: patrón
@Override
public Libro modificar(Long id, LibroPeticionDto dto) {
   Libro existente = obtenerPorId(id);   // si no existe -> 404
   validarLibro(dto);                    // si datos mal -> 400

   // comprobar duplicados ignorando el propio recurso
   for (Libro otro : libros) {
      if (!otro.getId().equals(id)
            && otro.getIsbn().equals(dto.getIsbn())) {
         throw new ConflictoException("ISBN duplicado");
      }
   }

   existente.setTitulo(dto.getTitulo());
   existente.setAutor(dto.getAutor());
   // ...resto de campos modificables
   return existente;
}
```

OJO	Si el enunciado dice que un campo NO se puede modificar, simplemente no hagas set de ese campo.

7. DELETE - Eliminar un recurso

CUÁNDO	El enunciado pide borrar/eliminar un recurso.
BUSCA	Ctrl + Shift + F -> public void eliminar. Referencia: Biblioteca Decroly Examen.

```
// ServiceImpl
@Override
public void eliminar(Long id) {
   Libro libro = obtenerPorId(id);  // 404 si no existe
   libros.remove(libro);
}

// Controller
@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable Long id) {
```

```
    libroServicio.eliminar(id);
    return ResponseEntity.noContent().build();
}
```

RESPUESTA	Borrado correcto -> 204 No Content. No devuelvas cuerpo.

8. GET con filtro - @RequestParam

CUÁNDO	El enunciado dice filtrar, buscar por, que contenga, desde, hasta, mínimo, máximo...
BUSCA	Ctrl + Shift + F -> EJEMPLO EXAMEN - VARIOS FILTROS OPCIONALES. Referencia: Biblioteca Decroly Examen.

```
// URL
GET /api/libros?titulo=quijote

// Controller
@GetMapping
public List<Libro> listar(
        @RequestParam(required = false) String titulo) {
    return libroServicio.filtrarPorTitulo(titulo);
}

// ServiceImpl: texto parcial sin distinguir mayúsculas
if (libro.getTitulo().toLowerCase()
        .contains(titulo.toLowerCase())) {
    resultado.add(libro);
}
```

REGLA	Filtro válido sin coincidencias -> 200 + []. No es 404.

9. Varios filtros, rangos y valores por defecto

BUSCA	Ctrl + Shift + F -> FILTRO POR RANGO. Referencia: Biblioteca Decroly Examen.
VARIOS	Usa @RequestParam(required = false). En el Service solo aplicas las condiciones cuyos parámetros no sean null.

```
@GetMapping
public List<Libro> listar(
        @RequestParam(required = false) String autor,
        @RequestParam(required = false) Integer anio,
```

```
    @RequestParam(required = false) Integer desde,
    @RequestParam(required = false) Integer hasta,
    @RequestParam(defaultValue = "0") int limite) {
  return libroServicio.filtrar(autor, null, anio, desde, hasta, limite);
}
```

RANGO	desde + hasta = valor entre un mínimo y un máximo. Valida que desde <= hasta; si no, 400.

```
if (desde != null && hasta != null && desde > hasta) {
  throw new DatosNoValidosException(
      "El mínimo no puede ser mayor que el máximo");
}
```

DEFAULT	@RequestParam(defaultValue = "...") sirve cuando el enunciado dice "si no se indica, usar...".

10. GET - Buscar por un campo único que no es el ID

CUÁNDO	ISBN, DNI, email, matrícula, código, número de serie... cualquier campo que identifique de forma única.
BUSCA	Ctrl + Shift + F -> BUSCAR POR CAMPO ÚNICO. Referencia: Biblioteca Decroly Examen.

```
// Controller
@GetMapping("/isbn/{isbn}")
public Libro obtenerPorIsbn(@PathVariable String isbn) {
  return libroServicio.obtenerPorIsbn(isbn);
}

// ServiceImpl: primero validar formato, luego buscar
if (isbn == null || !isbn.matches("\d{13}")) {
  throw new DatosNoValidosException("ISBN no válido");
}
for (Libro libro : libros) {
  if (libro.getIsbn().equals(isbn)) return libro;
}
throw new RecursoNoEncontradoException("No existe ese ISBN");
```

DISTINCIÓN	Formato incorrecto -> 400. Formato correcto pero no existe -> 404.

11. GET - Recursos relacionados

CUÁNDO	"Obtener los libros de una categoría", "alumnos de un curso", "productos de una marca"...

BUSCA	Ctrl + Shift + F -> listarPorCategoria. Referencia: Biblioteca Decroly Examen.

```
// Ruta jerárquica
@GetMapping("/{id}/libros")
public List<Libro> listarLibrosPorCategoria(@PathVariable Long id) {
    return libroServicio.listarPorCategoria(id);
}

// ServiceImpl
public List<Libro> listarPorCategoria(Long categoriaId) {
    categoriaServicio.obtenerPorId(categoriaId); // primero existe la categoría
    List<Libro> resultado = new ArrayList<>();
    for (Libro libro : libros) {
        if (libro.getCategoriaId().equals(categoriaId)) {
            resultado.add(libro);
        }
    }
    return resultado;
}
```

RESPUESTA	Categoría inexistente -> 404. Categoría existente sin libros -> 200 + [].

12. GET - Resultado calculado con DTO

CUÁNDO	El endpoint debe devolver resumen, total, estadísticas o datos combinados que no forman una entidad guardada.

BUSCA	Ctrl + Shift + F -> RESULTADO CALCULADO. Referencia: Biblioteca Decroly Examen.

17. Crea un DTO con exactamente los campos que quieres devolver.
18. Crea/usa un Service para obtener los datos necesarios y hacer los cálculos.
19. Construye el DTO en el Service.
20. El Controller solo devuelve ese DTO.

```
// DTO
@Getter
@AllArgsConstructor
public class CategoriaResumenDto {
    private Long categoriaId;
    private String nombre;
    private int numeroLibros;
    private int totalEjemplares;
}

// Controller
@GetMapping("/{id}/resumen")
public CategoriaResumenDto resumen(@PathVariable Long id) {
```

```
    return bibliotecaGestionServicio.obtenerResumenCategoria(id);
}
```

13. Cuando una operación necesita varios Services

CUÁNDO	Una regla necesita datos de varias entidades o varias partes de la aplicación.
BUSCA	Ctrl + Shift + F -> BibliotecaGestionServicio. Referencia: Biblioteca Decroly Examen.

```
@Service
@RequiredArgsConstructor
public class BibliotecaGestionServicio {
    private final CategoriaServicio categoriaServicio;
    private final LibroServicio libroServicio;

    // aquí combinas operaciones de ambos servicios
}
```

REGLA	No metas esa combinación en el Controller. Crea un Service coordinador si la lógica empieza a crecer.

14. Regla de negocio antes de realizar una operación

CUÁNDO	"Solo se puede... si...", "no se puede borrar cuando...", "no se puede modificar si...".
BUSCA	Ctrl + Shift + F -> REGLA AL BORRAR. Referencia: Biblioteca Decroly Examen.

```
public void eliminarCategoriaSiEstaVacia(Long id) {
    categoriaServicio.obtenerPorId(id); // 404 si no existe

    if (!libroServicio.listarPorCategoria(id).isEmpty()) {
        throw new ConflictoException(
                "No se puede eliminar porque todavía tiene libros");
    }

    categoriaServicio.eliminar(id);
}
```

409	Usa 409 cuando la petición es válida pero entra en conflicto con el estado actual: duplicado, borrar algo con dependencias, transición no permitida...

15. Errores - qué excepción usar

Situación	Excepción	HTTP
Dato mal formado, vacío, negativo, rango inválido	DatosNoValidosException	400 Bad Request
Se busca un recurso concreto que no existe	RecursoNoEncontradoException	404 Not Found
Duplicado o regla incompatible con el estado actual	ConflictoException	409 Conflict
API externa caída, solo si se trabaja ese bloque	ServicioExternoException	503 Service Unavailable

BUSCA Ctrl + Shift + F -> ManejadorGlobalExcepciones. Referencia: Biblioteca Decroly Examen.

```
@RestControllerAdvice
public class ManejadorGlobalExcepciones {

   @ExceptionHandler(RecursoNoEncontradoException.class)
   public ProblemDetail noEncontrado(RecursoNoEncontradoException ex) {
      return ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
   }
}
```

NO HACER No captures Exception.class "por si acaso". Puede ocultar errores reales de programación.

16. Tabla rápida de códigos HTTP

Código	Úsalo cuando...	Ejemplo
200	GET correcto o PUT que devuelve el recurso	GET /api/libros/1
201	POST crea correctamente	POST /api/libros
204	DELETE correcto sin cuerpo	DELETE /api/libros/1
400	Datos/parámetros inválidos	rango imposible, ISBN mal formado
404	Recurso concreto inexistente	ID 999
409	Conflicto con estado actual	ISBN duplicado, categoría con libros
503	Servicio externo no disponible, solo si se pide consumo externo	API externa caída

17. Endpoint que usa configuración

CUÁNDO El enunciado da valores configurables: nombre, máximo, días, IVA, límite, URL...

BUSCA	Ctrl + Shift + F -> BibliotecaPropiedades. Referencia: Biblioteca Decroly Examen.

```
# application.properties
biblioteca.nombre=Biblioteca Decroly
biblioteca.max-prestamos=3

@Component
@ConfigurationProperties(prefix = "biblioteca")
@Getter @Setter
public class BibliotecaPropiedades {
   private String nombre = "Biblioteca Decroly";
   private int maxPrestamos = 3;
}
```

REGLA	No hardcodees en Java valores que el enunciado identifica como configuración.

18. Swagger/OpenAPI - solo si lo pide el examen

SI NO LO PIDE	No añadas springdoc ni pierdas tiempo con @Tag, @Operation, @ApiResponse o @Schema.
SI LO PIDE	Primero haz funcionar el endpoint. Después documenta. La documentación no sustituye la lógica.

```
@Operation(summary = "Obtener un libro")
@ApiResponse(responseCode = "200", description = "Libro encontrado")
@ApiResponse(responseCode = "404", description = "Libro no encontrado")
@GetMapping("/{id}")
public Libro obtenerPorId(@PathVariable Long id) { ... }
```

19. Probar endpoints sin depender de Internet

GET	Puedes usar navegador o curl.exe.

```
curl.exe http://localhost:8080/api/libros
curl.exe http://localhost:8080/api/libros/1
curl.exe "http://localhost:8080/api/libros?desde=2000&hasta=2020"
```

POST/PUT	Con curl.exe envía Content-Type y JSON. En PowerShell puedes encerrar el JSON entre comillas simples.

```
curl.exe -X POST http://localhost:8080/api/categorias `
 -H "Content-Type: application/json" `
 -d '{"nombre":"Narrativa","descripcion":"Novelas"}'
```

```
curl.exe -X DELETE http://localhost:8080/api/categorias/1
```

POSTMAN

Úsalo si en el PC del examen funciona offline. No dependas de que un workspace cloud cargue.

20. Si algo falla: diagnóstico rápido

Síntoma	Revisa primero
404 en una ruta que debería existir	@RestController, @RequestMapping, @Get/Post/Put/DeleteMapping y que la clase esté bajo el paquete raíz.
400 inesperado	Body JSON, nombres de campos, validaciones del Service y tipos de @RequestParam/@PathVariable.
500	Consola de IntelliJ. Es un error interno; no cambies el status a ciegas.
409	Busca duplicados o una regla de negocio que impida la operación.
Spring no puede inyectar un Service	@Service en la implementación, constructor/@RequiredArgsConstructor y tipo de interfaz correcto.
Endpoint devuelve mal los datos	Comprueba si la lógica se ha metido en Controller en vez de Service, y revisa DTO/getters.
Maven intenta descargar algo sin Internet	No añadas dependencias innecesarias. Usa las versiones ya trabajadas/cacheadas en el PC de clase.

21. Hoja de decisión rápida

Frase típica del enunciado	Patrón
"Listar todos"	GET + listar()
"Obtener por id"	GET /{id} + obtenerPorId + 404
"Crear"	POST + @RequestBody + validar + 201
"Modificar"	PUT /{id} + obtener existente + validar + actualizar
"Eliminar"	DELETE /{id} + 204
"Filtrar/buscar por..."	@RequestParam + método filtrar en Service
"Entre X e Y"	dos @RequestParam + validar rango + comparar
"Si no se indica..."	@RequestParam(defaultValue = "...")
"Por ISBN/DNI/email/código"	GET por campo único + validar formato + 404
"Los X de un Y"	ruta /{id}/relacionados + comprobar padre + filtrar hijos
"Resumen/total/estadísticas"	DTO de respuesta + Service que calcula
"Solo se puede si..."	regla en Service + 409 si entra en conflicto

ÚLTIMA REGLA

No copies un endpoint entero sin entender qué cambia. Copia el patrón y adapta entidad, campos, reglas, ruta y errores.

22. Referencias offline para este manual

- Biblioteca Decroly Examen: proyecto catálogo validado con la carpeta EJEMPLOS EXAMEN de Postman.
- UD3 3.2 - API REST: capa Controller, rutas, métodos HTTP, parámetros y códigos de estado.
- UD3 3.3 - capa Service y configuración: separación de responsabilidades, inyección y propiedades.
- UD3 3.4 - documentación/consumo: OpenAPI y 503 solo cuando el enunciado trabaje esos bloques.
- GUIA_ENDPOINTS_OFFLINE.html: catálogo searchable de endpoints y ejemplos.

23. Checklist al terminar CADA endpoint

	Comprobación
[ ]	La ruta y el método HTTP coinciden con el enunciado.
[ ]	El Controller solo recibe/delega/responde.
[ ]	La lógica y validaciones están en Service.
[ ]	Los errores devuelven el código correcto.
[ ]	He probado un caso correcto.
[ ]	He probado al menos un caso de error.
[ ]	Si hay filtro sin resultados, devuelve 200 + [] cuando corresponda.
[ ]	Solo he añadido Swagger/Postman si lo pide el examen.